In-class check: Binary Search Rank – Answers

COSC 2230 (CS III) Fall 2026 · Augustana U. · Monday, October 26

The sorted array: `a = [ann, bob, bob, cat]` (indices 0 to 3)

Key	Returns	Why
`bob`	`1`	the smallest index of the two `bob` entries
`bo`	`-1`	not in the array; `bo` < `bob` (a prefix comes first), so the first larger entry is `a[1]`: return -1. This is $-i$ with $i = 1$, not the old "not found" value.
`dan`	`-4`	not in the array; nothing is larger, so $i =$ `a.length` $= 4$
`aaa`	`0`	not in the array; the first larger entry is `a[0]`: -0, which is 0 – it looks like a hit at index 0

4. Why not stop at the first `bob`? The first `mid` is 2, the *second* `bob`: the search from class returns 2. A match need not be the first copy of the key, so the first copy may still be to the left. The search has to keep looking in the lower half after a match and decide at the end, when the interval is empty.

Extra (`aaa`). `search()` returns 0, the same as a hit at index 0. A client cannot tell the two apart from the return value alone: it has to check `a[0].equals(key)` itself. That is why `Arrays.binarySearch()` returns $-i - 1$ on a miss.

The big ideas

1. **Don't stop at a hit.** With duplicates, the first `a[mid]` that equals the key can be any copy. To find the first copy, keep searching.
2. **Search for a boundary, not a key.** The helper no longer asks "where is the key?" but "where does $\geq$ key begin?".
3. **The invariant makes it correct:** `a[0, lo)` is $<$ key and `a[hi, N)` is $\geq$ key. It holds at the start, and at the end (`lo == hi`) `lo` is the boundary.
4. **Decide hit or miss once, at the end:**

```
if (i < a.length && a[i].equals(key)) return i;   // hit: first copy
return -i;                                        // miss: insertion point
```

5. **Still logarithmic**, even for all duplicates: $\sim$lg N compares.
6. $-0 == 0$ **for an `int`**, so `alice` and `bob` both return 0.
7. **Immutable keys:** binary search trusts that the array stays sorted. A key that can change after sorting breaks the order silently. A `String` can't change, so the order stays valid.